

Exercise Sheet: TCP and UDP with Scapy

Luca Campa

Introduction to Cybersecurity – University of Klagenfurt

Rules

- You can use any online resources, but you must write the code yourself. I'm aware of the usage of AI tools, but I expect you to understand, describe and justify your solution.
- You can ask for help from the instructor or teaching assistants during office hours.
- Please submit your code and a brief report (max 3 pages, excluding images, Wireshark screenshots, snippets of code etc..) describing your implementation and results. Upload a zip file containing a different folder for each exercise, with the corresponding code, a short guide on how to execute your scripts. The report should include:
 - A description of your implementation approach for each exercise.
 - An analysis of the results, including any challenges you faced and how you overcame them.

The report should NOT include:

- Your full scripts.
- If you want to include code snippets, make sure they are relevant to the discussion. You can refer to the code by using the line numbers in the scripts.

The scripts must be well-commented. Each function should have a clear description of type of inputs, outputs and what it does. The code should be organized and readable.

- You will be asked to present one of your solutions (not all the exercises) during the Friday Lab Sessions. Make sure to be prepared to explain your code and the results you obtained.

1 Environment and Requirements

- Use a Linux VM.
- Install Python-3 and the Scapy package:

```
python3 -m pip install --upgrade scapy
```

- Run scripts with administrative privileges (raw sockets need this):

```
sudo python3 your_script.py
```

- Use Wireshark to validate each step.
- After connection, if you get issues, check the status of your port with `ss -tuna | grep <port>`.

If you want to use TCPDUMP instead of Wireshark:

```
sudo tcpdump -n -i lo tcp or udp
sudo tcpdump -n -i <iface> host <target_ip> and \ (tcp or udp\)
```

2 Scapy Quick Guide (Read Before Coding)

- Protocol layering uses operator / (in other words, different layers of the protocol stack are separated by /), e.g. IP()/TCP()/Raw().
- `send()` sends at Layer 3 (IP)
- `sr1()` sends and waits for one response.
- `sniff()` captures packets with specified filters and callbacks.
- `RandShort()` gives a random source port.

Example

```
from scapy.all import IP, TCP, UDP, Raw, send, sr1, sniff, RandShort

# Build an IP/TCP SYN packet
syn = IP(dst="127.0.0.1")/TCP(sport=RandShort(), dport=3000, flags="S",
    seq=1000)

# Send and wait for single answer (e.g., SYN-ACK)
ans = sr1(syn, timeout=2, verbose=False)

# Build and send UDP payload
udp_pkt = IP(dst="127.0.0.1")/UDP(sport=40000, dport=9000)/Raw(load=b"
    hello")
send(udp_pkt, verbose=False)
```

TCP fields:

- **flags:** S (SYN), A (ACK), F (FIN), R (RST), P (PSH)
- **seq:** sequence number sent by who is sending the packet
- **ack:** acknowledgement number expected from the other side
- **Note:** As seen during the lecture, payload length affects ACK arithmetic: if payload has n bytes, ACK increases by n .

3 Setting up your VM to avoid interference

Your operating system has a normal TCP stack and it may generate RST packets, preventing your scripts from working correctly. To avoid this:

- filter outgoing RST packets temporarily,
- work on localhost (loopback) + dedicated ports (fix also the source port so that you can setup firewall rules to prevent interference).

Avoiding unexpected RST

1. Use a **fixed** client source port in your Scapy TCP scripts (for example `src_port = 40000`).
2. Add a temporary iptables rule that drops only matching outbound RST packets:

```
sudo iptables -I OUTPUT -p tcp --tcp-flags RST RST
-s 127.0.0.1 -d 127.0.0.1 --sport 40000 --dport 3000 -j DROP
```

3. Run the listening server: e.g. `nc -lnvp 3000`.
4. Run you scripts
5. Verify the exchanges with Wireshark or tcpdump.
6. Remove the temporary rule after finishing:

```
sudo iptables -D OUTPUT -p tcp --tcp-flags RST RST
-s 127.0.0.1 -d 127.0.0.1 --sport 40000 --dport 3000 -j DROP
```

4 Exercise 1: TCP Three-Way Handshake

By using Scapy, implement a client script that performs a full TCP handshake.

- Use localhost and open a TCP port (`nc -lnvp 3000` on another terminal).
- Choose:
 - a destination IP (e.g. 127.0.0.1 for localhost),
 - a destination port (e.g. 3000),
 - a random source port (since kernel module can interfere, I suggest fixing it for testing, e.g. 40000),
 - an initial client sequence number x .

What to do

1. Send SYN with `seq = x`.
2. Receive SYN-ACK from server with `seq = y`, `ack = x+1`.
3. Validate both conditions:
 - server flags contain SYN and ACK,
 - server ACK equals your SYN sequence plus 1.
4. Send final ACK with:
$$\text{seq} = x + 1, \quad \text{ack} = y + 1$$
5. Perform graceful connection termination:
 - client sends FIN,ACK,
 - server responds with ACK (or combined FIN,ACK),
 - if FIN is separate, wait for server FIN,
 - client sends final ACK.

Output and validation

For debugging and for evaluation, print a clear trace of the handshake, including:

- sent SYN (seq, sport, dport)
- received SYN-ACK (seq, ack, flags)
- sent ACK (seq, ack)
- handshake successful / failed (with reason)
- sent FIN and received FIN/ACK sequence
- final ACK sent.

Hints:

- Use `sr1()` for SYN, then `send()` for final ACK.
- Set a `timeout` to avoid hanging forever.
- Use `pkt.show()` when debugging packet fields.

5 Exercise 2: TCP Data Exchange

This exercise continues from **Exercise 1**. Here, you are asked to send at least one payload segment before the connection is closed, and to verify the ACK progression w.r.t the payload length.

What to do

1. After connection establishment, send payload (for example "HELLO WORLD").
2. Receive ACK from server and verify:

$$\text{ack_from_server} = \text{client_seq} + \text{payload_length}$$

3. (Optional) perform more exchanges and verify ACK progression for each segment.

6 Exercise 3: UDP Sender/Receiver

In this exercise, you will implement a simple UDP sender and receiver using Python sockets. You will also experiment with packet loss and reordering to understand the behavior of UDP.

The receiver

A pure packet sniffer does not open UDP port state in the kernel. If no UDP socket is bound, sender traffic can trigger ICMP *Destination unreachable (Port unreachable)* and your application-level receiver logic may fail. Hence, this is an example of a simple working receiver:

```
#!/usr/bin/env python3
import socket

LISTEN_IP = "127.0.0.1"
LISTEN_PORT = 9000

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((LISTEN_IP, LISTEN_PORT))
```

```
print(f"[*] Listening on {LISTEN_IP}:{LISTEN_PORT}")
while True:
    data, addr = sock.recvfrom(65535)
    print(f"[RX] from {addr[0]}:{addr[1]} -> {data.decode(errors='replace')}")
```

What to do

1. Write a sender that transmits $n > 10$ datagrams with sequence IDs in payload, e.g. `msg-1`, ..., `msg-n`.
2. Print the received messages on the receiver side.
3. Simulate packet reordering by
 - sending the datagrams in a non-sequential order (for example, send `msg-1`, then `msg-3`, then `msg-2`);
 - improving the receiver so that it can buffer out-of-order messages and print them in the correct order (remember that this operation is done at the application level).
4. Simulate packet loss by randomly dropping some datagrams in the sender (for example, use `random.random() < 0.3` to drop 30% of the messages).
5. Change the behaviour of the receiver to detect missing messages and print a warning when a gap in the sequence IDs is detected.
6. (Optional) Implement a simple retransmission mechanism in the sender: if you detect a gap in the sequence IDs at the receiver, send a request for retransmission of the missing message, and update in the sender to handle this request and resend the specified datagrams.

7 Exercise 4: A simple NMAP TCP SYN Scan

In this exercise, you will implement a simple TCP SYN scan using `Scapy`. The goal is to determine if a specific TCP port on a target host is open, closed, or filtered.

What is NMAP?

NMAP (Network Mapper) is a free, open-source utility used to discover hosts, map networks, and perform security auditing. It sends packets to target systems and analyzes the responses to identify active devices, open ports, operating systems, and potential vulnerabilities. If you want to learn more about NMAP, check the official website: <https://nmap.org/>.

What to do

1. Choose a target IP and a TCP port to scan (for example, localhost and port 80).
2. Send a TCP SYN packet to the target port.
3. Analyze the response:
 - If you receive a SYN-ACK, the port is open.
 - If you receive a RST, the port is closed.
 - If you receive no response after a timeout, the port is filtered (e.g., by a firewall).
4. Print the result of the scan (open/closed/filtered).